

ЛР 2 Скриптинг ТЗ

1. Цель

Встроить в ваш движок **скриптинг как подсистему** и реализовать на скриптах **одну реальную игровую механику целиком** так, чтобы правила игры менялись без перекомпиляции и без перезапуска движка.

Лабораторная проверяет два умения: провести осмысленную границу "C++/скрипты" в своём движке и сделать итерацию над геймплеем живой (hot reload).

2. Обязательный состав

2.1. Встраивание скриптового языка

- **Язык на выбор команды:** Python (pybind11), Lua (sol2), C# (CoreCLR hosting) или иной на ваш страх и риск.
- Язык встраивается **как подсистема движка**, а не "вызов в одном месте": явные инициализация/шатдаун интерпретатора (кто и когда создаёт state, когда он умирает), единая точка загрузки и выполнения скриптов, понятное время жизни state'a (что в него попадает и когда).
- Дистрибуция интерпретатора продумана: рантайм поставляется рядом с билдом (для Python 3.11 embeddable package - ≈10 МБ) или осознанно ищется в системе - на приёмке спрошу, что будет на машине "пользователя".

2.2. Bindings двусторонние

- **Движок -> скрипт:** entry points механики (инициализация, update на кадр/тик; реакция на события движка, если делаете события, см. допфичи).
 - Каноническая форма - скриптовый класс поведения, расширяющий C++-базу (для Python/pybind11 - trampoline).
- **Скрипт -> движок:** API, достаточный для механики из 2.4 - не "две функции для галочки". Минимум 4 категории: работа с сущностями/трансформом, ввод, спавн/уничтожение, запросы к миру (что есть в движке).
- **Владение объектами на границе продумано:** скрипт не держит висячих ссылок на C++-объекты и наоборот.
- Ручных bindings достаточно для зачёта; генерация bindings и stubs - в допфичах (см. 4).

2.3. Hot reload кода

- Перезагрузка скрипта **без перезапуска движка** (файловый вотчер или явная команда на ваш выбор). Демонстрируется вживую: поменяли число или условие в файле -> поведение изменилось при следующем запуске playmode.
- Минимум: перезагрузка вне playmode. Иное в допфичах см 4.

2.4. Механика целиком на скриптах

- **Одна механика игры**, реализованная целиком на скриптах: логика, состояние, реакция на события.
- **Реальность ваших движков:** готовых механик почти ни у кого нет (есть движение, коллизии, камера, спавнеры частиц, триггеры) - значит, механику строим небольшую, но настоящую.

- Кандидаты: поведение противника / волны спавна; оружие и стрельба; ловушки, триггеры, двери, кнопки; подбор предметов / инвентарь; правила режима (таймер, счёт, условия победы)
- "Целиком" = **игровые решения принимает скрипт**. Движок предоставляет сервисы (ввод, запросы коллизий, рендер, события), но не логику механики. Скрипт-обёртка над механикой, живущей в C++, не засчитывается.
- **Живой критерий на приёме**: я прошу изменить правило механики (число, условие, порядок) - вы правите скрипт на месте, reload, изменение видно в игре. Готовьте механику, где это наглядно.

2.5. Шаблонный спавн (prefab)

Скриптовая механика не живёт в вакууме: ей нужно **порождать объекты**. Поэтому в состав лабораторной входит опорная часть gameplay-слоя - спавн из шаблонов.

- **Data-driven описание шаблона объекта**: набор компонентов + начальные значения в файле (JSON/YAML), спавн экземпляра из скрипта одной командой.
- Начальные значения включают **поля скриптовых поведений**: настраиваемые параметры механики (числа, флаги, строки - баланс) задаются данными шаблона, а не константами в коде скрипта.
- Механика из 2.4 обязана **пользоваться** шаблонным спавном (волна противников, снаряды, ловушки - по смыслу механики).

2.6. Правка полей в редакторе (минимум)

- Выделили объект в редакторе -> видны поля его скриптового поведения с текущими значениями -> значение можно поменять -> правка сохраняется в сцену/шаблон и применяется при следующем спавне/play.
- *Реальность ваших движков*: если редактора (ПЗ 5 прошлого семестра) нет - достаточно минимальной панели ImGui: выбор объекта из списка + правка полей его скриптового поведения. Это часть лабораторной, а не допфича.

3. Критерии приёмки

Базовый вариант: **16 + 2 балла**

Блок	Баллы	Как проверяется
Наличие и полнота	8	Демо вживую: скриптовая подсистема работает (2.1); двусторонние bindings с осмысленным владением (2.2); hot reload показан на ходу (2.3); механика из 2.4 играется, правило меняется по просьбе преподавателя без перезапуска; механика спавнит объекты из шаблонов (2.5); поля поведения правятся в редакторе и сохраняются (2.6)
Стабильность	4	Сценарии вживую: сломанный скрипт + reload - движок жив, в логе файл:строка; серия reload'ов подряд; длинная сессия без крашей и утечек state'a; корректный выход. Падение на скриптовой ошибке = минус баллы этого блока
Допфичи	4	Любая работа над движком сверх обязательного (см. 4)
Своевременность	+2	Демо в свой приёмочный демо-день (16.10)

Приёмка - живое демо + презентация.

4. Допфичи (примеры, не исчерпывающе)

Сквозное правило семестра - job system из ЛР 1:

- hot reload через job system: файловый вотчер / перечтение / валидация скрипта - задачей в пуле, применение результата - на главном потоке;

Bindings и граница:

- генерация bindings (по аннотациям, заголовкам или описанию API) вместо ручных;
- **поддержка IDE для скриптеров:** у скриптера работают автодополнение и подсказки типов (для Python — stubs через pybind11-stubgen или руками)
- **события движка в скрипте:** подписка скрипта на события (столкновение, триггер, таймер), обработчики на скриптовой стороне; глубже - очередь событий с приоритетами и постингом «в будущее», подписка/отписка на ходу. У кого шины событий нет - минимальный event bus (тип + аргументы + подписчики) строится здесь как допфича

Hot reload в Play режиме:

- L1: сброс состояния объекта со скриптом
- L2: перенос состояния на новый объект (тех полей, которые не изменились)

Редактирование полей и ссылки на объекты (поверх минимума из 2.6):

- L1: **инспектор полей, полный** - скрипт объявляет редактируемые поля с типами и значениями по умолчанию, редактор показывает виджет по типу (слайдер/чекбокс/выпадающий список), валидация значений;
- L2: **поля-ссылки на игровые объекты** - поведение ссылается на другой объект (кнопка → дверь): назначение ссылки в инспекторе, сериализация как id/handle, корректная обработка уничтожения цели (stale handle → null, а не висячий указатель).

Отладка и наблюдаемость:

- REPL-консоль в игре (выполнить скриптовый код вживую, посмотреть состояние);
- профилирование: время скриптовых entry points зонами Trasy - на трейсе видно, сколько стоит скриптовый тик;
- подключаемый отладчик скриптов (для Python - debugpy, для Lua - mobdebug или аналог);
- изоляция/sandbox: лимиты на скрипт, отдельный state на мод/сущность.

Мир и данные сверх обязательного:

- **загрузка уровня в рантайме** (смена сцены на ходу, а не только стартовая): чтение и парсинг сцены - задачами job system из ЛР 1 (сквозное правило), финализация на главном потоке, скрипты механики корректно переживают смену уровня;
- **save game** (сохранение состояния, не только сцены): сериализуются и скриптовые состояния механик;
- **запросы к миру из скрипта:** raycast / proximity / "все объекты типа X в радиусе" - если механика требует "видеть" мир (ИИ, ловушки, прицеливание).

5. Формат приёмки

- **Промежуточное демо 09.10** (нед. 6): показ прогресса + ротационные вопросы.
- **Приёмка 16.10** (нед. 7): показ живьём - 3 минуты демо + вопросы.
- **Ротационные вопросы (АИД-2)**: отвечают двое от команды - кого укажу я, не презентатор.
Вопросы - по всему коду команды и лекциям.